Reporte: p1 ejercicio 14

En este programa evolucionamos el concepto de la calculadora aplicando Programación Orientada a Objetos, herencia y recursividad. La clase base CalculadoraBase gestiona los datos de entrada principales y la suma, mientras que la clase derivada CalculadoraNueva hereda sus miembros y añade la resta, además de métodos recursivos para multiplicar, elevar a potencia, dividir, calcular el factorial y obtener el número de Fibonacci.

Código Completo:

```
#include <iostream>
using namespace std;

class CalculadoraBase {
protected:
    int num1, num2;

public:
    CalculadoraBase(int n1, int n2) {
        num1 = n1;
        num2 = n2;
    }

    int sumar() {
        return num1 + num2;
    }
};

class CalculadoraNueva : public CalculadoraBase {
public:
    CalculadoraNueva(int n1, int n2)
        : CalculadoraBase(n1, n2) {}

    int restar() {
        return num1 - num2;
    }

    // Multiplicacion recursiva por sumas sucesivas
    int multiplicar(int a, int b) {
        if (b == 0) {
            return 0;
        }
        if (b < 0) {
            return -multiplicar(a, -b);
        }
        return a + multiplicar(a, b - 1);
    }

    // Potencia recursiva por multiplicaciones sucesivas
    int potencia(int base, int exponente) {
        if (exponente == 0) {
            return 1;
        }
        if (exponente < 0) {
            cout << "Error: el exponente debe ser cero o positivo.\n";
```

```
            return 0;
        }
        return base * potencia(base, exponente - 1);
    }

    // Division entera recursiva por restas sucesivas
    int dividir(int a, int b) {
        if (b == 0) {
            cout << "Error: division por cero.\n";
            return 0;
        }
        int resultado = dividirPositivo(
            (a < 0 ? -a : a),
            (b < 0 ? -b : b)
        );
        if ((a < 0) != (b < 0)) {
            resultado = -resultado;
        }
        return resultado;
    }

private:
    int dividirPositivo(int dividendo, int divisor) {
        if (dividendo < divisor) {
            return 0;
        }
        return 1 + dividirPositivo(dividendo - divisor, divisor);
    }

public:
    // Factorial recursivo: n! = n * (n-1)!
    int factorial(int n) {
        if (n < 0) {
            cout << "Error: el factorial no existe para negativos.\n";
            return 0;
        }
        if (n == 0 || n == 1) {
            return 1;
        }
        return n * factorial(n - 1);
    }

    // Fibonacci recursivo
    int fibonacci(int n) {
        if (n < 0) {
            return -1;
        }
        if (n == 0) {
            return 0;
        }
        if (n == 1) {
            return 1;
        }
        return fibonacci(n - 1) + fibonacci(n - 2);
    }
};

int main() {
```

```
    int n1, n2, n;

    cout << "--- CALCULADORA RECURSIVA ---\n";
    cout << "Introduce el primer numero: ";
    cin >> n1;
    cout << "Introduce el segundo numero: ";
    cin >> n2;

    CalculadoraNueva calc(n1, n2);

    cout << "\nResultados de la calculadora:\n";
    cout << "Suma: " << calc.sumar() << endl;
    cout << "Resta: " << calc.restar() << endl;
    cout << "Multiplicacion recursiva: "
         << calc.multiplicar(n1, n2) << endl;

    if (n2 >= 0) {
        cout << "Potencia recursiva: "
             << calc.potencia(n1, n2) << endl;
    } else {
        cout << "Potencia: el exponente debe ser cero o positivo.\n";
    }

    cout << "Division entera recursiva: "
         << calc.dividir(n1, n2) << endl;

    cout << "\nIntroduce un numero para factorial y Fibonacci: ";
    cin >> n;

    if (n >= 0 && n <= 12) {
        cout << "Factorial de " << n << ": "
             << calc.factorial(n) << endl;
        cout << "Fibonacci en la posicion " << n << ": "
             << calc.fibonacci(n) << endl;
    } else if (n > 12) {
        cout << "El numero es muy grande para mostrar resultados seguros con int.\n";
    } else {
        cout << "Factorial y Fibonacci requieren un numero no negativo.\n";
    }

    return 0;
}
```

Explicación del Código:

1. Estructura de Herencia (CalculadoraBase y CalculadoraNueva)
CalculadoraBase define los atributos protegidos (protected: int num1, num2) y el método sumar(). CalculadoraNueva hereda de ella y reutiliza su constructor mediante la lista de inicialización : CalculadoraBase(n1, n2).

2. Operaciones Aritméticas Recursivas (multiplicar, potencia y dividir)
En lugar de ciclos for o while, los métodos se auto-invocan reduciendo el problema hasta un caso base:
• multiplicar(a, b): Retorna a + multiplicar(a, b - 1) hasta que b es 0.
• potencia(base, exp): Retorna base * potencia(base, exp - 1) con caso base exp == 0 (retorna

1).
• dividir(a, b): Utiliza el método auxiliar privado dividirPositivo que resta el divisor sucesivamente (1 + dividirPositivo(dividendo - divisor, divisor)) y ajusta el signo al final.

3. Algoritmos Clásicos: Factorial y Fibonacci
• factorial(n): Calcula n! mediante n * factorial(n - 1) teniendo como caso base 0! = 1! = 1.
• fibonacci(n): Retorna el n-ésimo término sumando fibonacci(n - 1) + fibonacci(n - 2) con casos base para n = 0 y n = 1.

4. Ejecución Principal (main) y Validaciones
Solicita los números iniciales, crea el objeto 'calc' e imprime los resultados básicos. Posteriormente solicita un entero adicional para calcular factorial y fibonacci, agregando una validación (n <= 12) para evitar el desbordamiento de memoria o de tipo entero (int).